Deep Dive 08 | End-to-End Execution Trace

Walk through the most important user journeys and diagnose failures by boundary.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Boot sequence

- Start MongoDB Atlas access through MONGO_URI, then launch Express from backend. dotenv loads configuration, Mongoose connects, middleware and routes mount, and the server listens on PORT.
- Start frontend with Vite; React mounts AuthContext/ToastContext and App routes.
- Start Flask independently with its Gemini key if chatbot interaction is required.
- Open localhost:5173; Vite proxies API and uploads paths to Express.

2. Account journey

A visitor opens registration. React sends FormData to POST /api/auth/register. Express validates required fields, checks duplicates, stores an optional upload path and creates a Mongoose User. The pre-save hook hashes the password. The route signs a one-hour JWT and returns user data. AuthContext stores that data locally and the user reaches the dashboard. On later protected calls the browser sends a Bearer token; middleware verifies it and loads the user.

3. Alerts journey

The student dashboard renders AlertTicker. It fetches /api/alerts. Express calls the NDMA SACHET summary endpoint, maps source records to a common shape, sorts and returns a limited list. The ticker loads each CAP detail through /api/alerts/cap/:id; Express fetches XML, parses it and returns readable fields. The UI chooses a clear human-readable alert title and links to the detail page.

4. Nearby-service journey

The user opens /dashboard/student/nearby. Browser geolocation supplies coordinates; if denied, the page switches to the documented fallback. React requests /api/nearby. Express validates latitude, longitude and radius; serves a fresh cached result, joins a pending request or calls Geoapify. The response is converted to frontend elements. React filters and calculates distances. NearbyMap adds OSM tiles, user marker and place markers.

5. Chat and game journeys

Chat: the user submits a prompt, React posts JSON to Flask /chat, Flask constructs the Gemini request and returns generated text. Drill: the page loads the Phaser HTML and game script; scene transitions, keyboard controls, hazards, collision logic and result UI all run in-browser.

6. Failure diagnosis by boundary

Symptom	Check first	Next check
Dashboard route 404	Vite/React Router fallback.	Vercel rewrite or Express SPA handler.
Login fails	Network status and JSON error.	Express logs, Mongo connection, password hash, JWT secret.
Google callback fails	Callback URL and client credentials.	FRONTEND_URL, Passport config, session cookie/proxy.
No nearby places	Backend response status and error.	Key format/restrictions/quota, provider response, coordinates/cache.
Alert ticker empty	/api/alerts payload and error field.	SACHET availability, CAP identifier and parser shape.
Chatbot errors	VITE_FLASK_URL and browser network.	Flask logs, Gemini key/quota, timeout/CORS.
Map appears blank	Leaflet container size and tile request.	Tile network, map init lifecycle, CSS height.

7. End-to-end reasoning checklist

- Name the caller, endpoint, service, data store/provider and returned data.
- Check status code and response shape before investigating UI rendering.
- Separate configuration failures from code defects and provider outages.
- Use safe diagnostic logs and never reveal secret values.
- Verify the deployed revision and environment, not only local code.
- Keep the user-facing message honest about staleness, fallback location and external data coverage.

Practice task: choose an observed failure and write a short incident trace: symptom, boundary, evidence, root cause, user impact, fix, and verification. Do not assume a local fix has reached production until the deployed revision is confirmed.